OAuthアプリを介した攻撃活動に対するGoogle Workspace監査ログの検知可能性の検証

赤岩 七海[1,3,a)]　矢部 海人[1]　瀧田 愼[1]　平薮 将志[1]　Thein Tharaphe Thin[1]　葛野 弘樹[1]
毛利 公美[2]　古本 啓祐[3]　高橋 健志[3]　白石 善明[1,b)]

概要：組織のセキュリティ監視は，利用者端末やサーバ上の挙動を見るエンドポイント検知に加え，クラウド上の API 操作を監査ログで監視することも不可欠である．OAuth 同意で得たトークンによる操作は端末を経由せずクラウド API として実行されるため，エンドポイント上には挙動が現れず，監査ログを用いても悪性アプリの操作であるかを判別できるとは限らない．本研究は，組織ユーザが Google 未承認の OAuth アプリに同意した状況を想定する．管理者が監査ログを用いて，OAuth アプリの利用をどこまで検知できるか，および，OAuth アプリの操作が悪意あるものかをどこまで判別できるかという検知可能性を検証する．Google Workspace において自作の OAuth アプリにより連絡先，Drive，Gmail，および Calendar 操作を実行する検証実験を行い，ログを分析した．評価の結果，監査ログは OAuth アプリの利用有無を示すが，一覧取得や閲覧の操作内容は記録が残らず，記録が残る操作においても悪意ある操作であるかを判別できないことを示した．

キーワード：OAuth，Google Workspace，監査ログ，SaaS セキュリティ，攻撃検知

Detectability of Attack Activities via OAuth Applications in Google Workspace Audit Logs

Nanami Akaiwa[1,3,a)]　Kaito Yabe[1]　Makoto Takita[1]　Masayuki Hirayabu[1]
Thin Tharaphe Thein[1]　Hiroki Kuzuno[1]　Masami Mohri[2]　Keisuke Furumoto[3]
Takeshi Takahashi[3]　Yoshiaki Shiraishi[1,b)]

Abstract: Organizational security monitoring requires not only endpoint detection, which observes behavior on user devices and servers, but also monitoring of API operations in the cloud through audit logs. Because operations performed with tokens obtained via OAuth consent are executed as cloud API calls without passing through the endpoint, such behavior does not appear at the endpoint, and audit logs alone do not necessarily allow administrators to determine whether an operation originates from a malicious application. This study assumes a situation in which an organizational user has consented to an OAuth application unapproved by Google. The extent to which an administrator can detect the use of the OAuth application through audit logs, and the extent to which the administrator can determine whether an operation performed by the OAuth application is malicious, are examined.A verification experiment was conducted in which a self-developed OAuth application performed operations on Contacts, Drive, Gmail, and Calendar within Google Workspace, and the resulting logs were analyzed. The evaluation results show that the audit log indicates whether an OAuth application was used, but no record remains for listing or viewing operations, and even for operations that are recorded, whether the operation is malicious cannot be determined.

Keywords: OAuth, Google Workspace, audit log, SaaS security, attack detection

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

組織のセキュリティ監視は，利用者端末やサーバ上の攻撃者による不審な挙動を捉えるエンドポイント検知（EDR/XDR）が広く用いられる．一方，Google Workspace[1] 等の SaaS では，ユーザが OAuth アプリに同意し，読み取りおよび書き込みの権限 (スコープ) を付与すると，アプリはクラウド上の API として操作を実行し，端末を経由しない．端末を経由しない操作はエンドポイント検知の対象に含まれないため，API 操作を監査ログで監視する必要がある．OAuth 認可フローで取得したトークンを用いた API を悪用し，金銭目的の攻撃の自動化に利用されることが報告されている [2].

米国 Cyber Safety Review Board の報告 [3] によれば，Microsoft Exchange Online への侵入を自組織の監査ログから検知できた組織は，監査ログの強化機能（Audit Premium）を有効化していた組織に限られていた．Audit Premium により，メールボックス内のメールアイテムへの実際のアクセスを記録する MailItemsAccessed イベントが取得可能となり，認証成功の有無だけでなく，メール内容への到達有無まで監査ログ上で判別できるようになる．強化ログを有効化していない組織は，侵入発覚後に機能を有効化しても過去のアクセス記録は取得できず，有効化以降のデータのみ取得できた．監査ログによる検知の範囲は，クラウドサービス事業者が当該イベントを記録する仕組みを実装しているか，および実装されたログを顧客がライセンス契約上取得できるかの両方に依存する．文献 [3] の事例は監査ログさえあれば検知できるとは限らないこと，記録範囲の設計自体が検知可能性を左右することを示す．

Balash[4] らは，多くのユーザがサードパーティアプリの権限を確認していないことを指摘しており，個人情報の閲覧という侵襲性の低い権限には高い懸念を示す一方，メール，カレンダー，およびクラウドストレージの変更・閲覧という侵襲性の高い権限には懸念が低いことを示している．ユーザ自身による権限確認が十分に機能しない以上，攻撃の検知は管理者側の監査ログ運用に委ねられるが，検知可能な範囲は明らかではない．

本研究の目的は，組織内のユーザが未承認の OAuth アプリに同意した状況を想定し，管理者が監査ログを用いて OAuth アプリの利用をどこまで検知できるか，また，OAuth アプリの悪意ある操作をどこまで判別できるかを明らかにすることである．そのため，組織内のユーザが Google 未承認の OAuth アプリの権限要求を認可することで発生する，付与スコープの範囲内での攻撃活動を想定したシナリオを構築する．検証実験において，検証実験用の Google Workspace テナントに対し自作の OAuth アプリを用い，連絡先，Drive，Gmail，および Calendar の操作を付与スコープの範囲内で API として実行し各操作が監査ログに記録されるかを確認する．通常操作と攻撃操作を区別する情報が含まれているかを分析し，OAuth アプリの利用検知および悪意判別それぞれの可能性を明らかにする．

本研究の貢献は次のとおりである．

- Google Workspace テナント上で，検証実験用に自作した未承認 OAuth アプリ (以下「検証実験用アプリ」) を用いて操作を実行し，管理コンソール UI 出力，および Admin SDK Reports API[5] (以下「Reports API」) 出力を操作カテゴリ単位で比較し，監査ログの記録範囲および表示粒度に差異が存在することを示した．
- OAuth アプリ経由の操作と，同一操作をユーザが手動で実施した場合をフィールド単位で比較し，操作の実行主体が OAuth アプリであるかユーザ本人であるかを識別可能なフィールド（アクターのアプリ情報，クライアントの種類，および OAuth プロジェクト ID）を明らかにした．
- 記録範囲および表示粒度の差異，および実行主体が識別可能なフィールドの有無を明らかにした結果を踏まえ，記録が存在する操作についても，悪意の有無を判別できないという検知の限界を明らかにした．

  組織が講じるべき対策として，認可時点での制御，Reports API を用いた監査，異常検知のためのアラート条件の設定，アプリアクセス無効化の自動化，およびユーザからの申告制度の作成を示した．

本稿の構成は次のとおりである．2 章では OAuth の認可フローと認可モデルについて述べ，3 章で脅威モデルと攻撃シナリオを説明する．4 章で検証実験環境の設計と構築について述べ，5 章で評価結果を示す．6 章で考察を述べ，7 章で関連研究との比較を行い，8 章でまとめと今後の課題を述べる．

2. OAuth の認可フローと認可モデル

OAuth 2.0 [6] の認可フローを図 1 に示す．OAuth 2.0 は，リソース所有者の認可に基づき，サードパーティアプリが保護されたリソースへ限定的にアクセスすることを可能にする認可フレームワークである．代表的な認可方式である Authorization Code Grant [7] では，ユーザが要求されたスコープの内容を確認したうえで認可操作を行うと，認可サーバは認可コードを発行する．サードパーティアプリは，取得した認可コードを認可サーバへ送信してアクセ

[1] 神戸大学大学院 工学研究科
Graduate School of Engineering, Kobe University
[2] 近畿大学 情報学部/情報学研究所
Faculty of Informatics/Cyber Informatics Research Institute, Kindai University
[3] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
a) akaiwa.nanami@gsuite.kobe-u.ac.jp
b) zenmei@port.kobe-u.ac.jp

© 2026 Information Processing Society of Japan

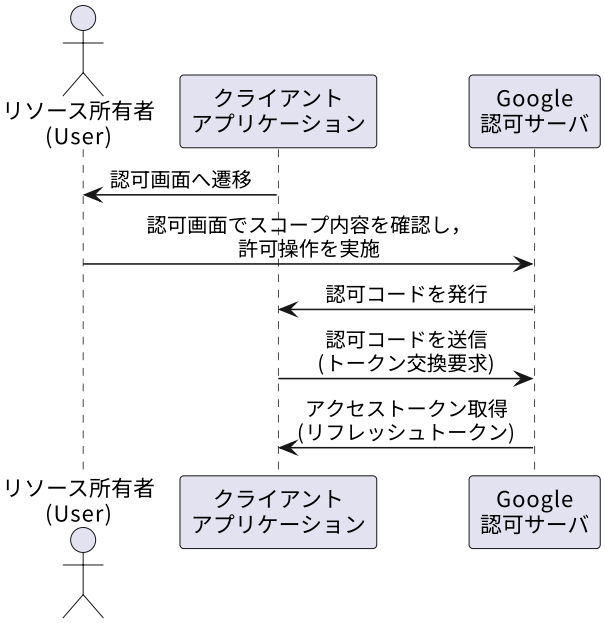


図 1: OAuth の認可フロー

ストークンとの交換を要求し，Google 認可サーバからアクセストークンを取得する．本研究は，同意操作の完了によりアクセストークンが発行された時点以降の Google API 呼び出しおよび監査ログへの記録状況を調査対象とする．

Google Workspace におけるサードパーティの OAuth アプリの API アクセスは，管理コンソールの API アクセスレベル設定に依存する．アクセスレベルが「制限なし」に設定された組織では，管理者による個別承認を経ることなく，ユーザ自身の認可操作のみで OAuth アプリへの認可が行われる．

3. 脅威モデルと攻撃シナリオ

3.1 ユーザモデル

- **ユーザ**：Google Workspace Business Starter を契約する「エージェントセキュリティ研究所」に所属し，連絡先，Gmail，Drive，および Calendar を含むアプリを業務で利用する．ユーザ 1 およびユーザ 2 を，同一の Google Workspace テナントに所属するユーザとして設定する．ユーザ 1 およびユーザ 2 は管理者権限を保有せず，一般利用者としての権限を持つ．
- **リソース**：ユーザが保有するリソースとして，攻撃対象となり得るデータを検証環境内に配置する．具体的には，Drive のマイドライブにドキュメントファイルおよびスプレッドシートを各 1 件配置し，連絡先にユーザ 2 のメールアドレスを登録し，カレンダーには予定を 2 件登録する．

  リソースは，OAuth アプリを介した攻撃シナリオにおいて，情報窃取や不正操作の対象として想定される典型的なデータ種別を模擬したものである．
- **デフォルト設定**：Google Workspace 管理コンソールにおける API アクセスレベルは「制限なし」であり，サードパーティの OAuth アプリはユーザの認可のみで各サービスへアクセス可能な状態である．加えて，「未設定のサードパーティ製アプリ」項目は，デフォルトで「サードパーティ製アプリへのアクセスをユーザに認可する」に設定されており，管理者による事前承認を必要としない．

  Google 公式ドキュメント [8] によると，Gmail データにアクセスし全世界で 100 人を超えるユーザを持つ未承認のサードパーティのアプリは制限の対象となり，管理者が当該アプリを信頼済みに指定しない限り，新規インストールには制限が適用される．

一般的な組織利用者を想定し，OAuth アプリによるスコープ確認画面の内容を十分に確認せず承認する可能性など，ユーザの判断に起因する限界を前提とする．

3.2 攻撃モデル

本研究では，組織外の第三者が未承認の OAuth アプリを用意し，組織内ユーザに正規の認可画面で権限付与を求める状況を想定する．ユーザが認可した後，攻撃者は付与されたスコープの範囲内で，次の Google API を呼び出す．攻撃者は，何らかの手段でユーザのメールアドレスを事前に入手するものとする．

- 連絡先の読み取り：連絡先一覧を取得する．
- メール送信：取得した宛先に対してユーザアカウントからメールを送信する．
- Drive への読み書き：Drive 上のファイル一覧とメタデータを取得したうえで，任意のファイルの内容を書き換え，攻撃者のサーバへエクスポートする．また，新規フォルダを作成し，ファイルをアップロードする．
- カレンダーの閲覧：ユーザのカレンダー上の予定を閲覧する．

3.3 攻撃シナリオ

攻撃者は，Google アカウント連携機能を利用するアプリとして，OAuth クライアントを作成する．要求するスコープは，連絡先の読み取り，メール送信，Drive への読み書き，およびカレンダーの閲覧を含む．攻撃者は，ユーザのメールアドレス宛にアプリの利用を促し，Google の認可画面へユーザを誘導する OAuth 認可画面の URL を送付する．ユーザが URL へアクセスし，表示された要求スコープの内容を確認したうえで，許可操作を行うことで，攻撃が開始される．認可操作の完了により，攻撃者はアクセストークンを取得する．攻撃者は取得したトークンを用いて，Google Workspace の各 API の呼び出しが可能となる．

攻撃者は，連絡先一覧を取得し，取得した宛先に対してユーザアカウントからメールを送信する．続いて，Drive

© 2026 Information Processing Society of Japan

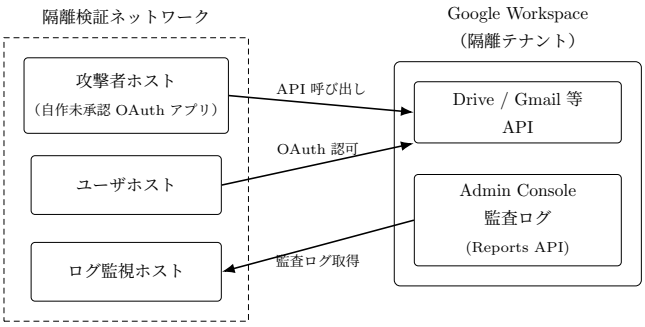


図 2: 検証環境の構成

上のファイル一覧とメタデータを取得する．任意のファイルの内容を書き換え，攻撃者のサーバへ PDF 形式でエクスポートする．さらに，新規フォルダを作成し，ファイルをアップロードする．最後に，ユーザのカレンダー上の予定を閲覧する．

4. 検証実験環境の設計と構築

4.1 設計方針と設計方法

3.3 節の攻撃シナリオに基づき，以下の 3 点を設計方針として環境を構築する．

（1）実環境の利用：本研究では模擬環境ではなく本物の Google Workspace テナントを用い，実際の攻撃に近い条件下での検証を実施する．

（2）監査ログの並行取得：Google Workspace が記録する監査ログの収集を，Reports API を通じた取得と，管理コンソール UI 上での検索結果エクスポートの 2 つの経路により行う．それぞれの経路において記録内容や項目に差異が存在するかを検証する．

（3）攻撃操作の自動化：攻撃手順を，認可 URL の生成，トークン交換，API 呼び出しという一連の流れとして，Python スクリプトにより実装する．スクリプトによる自動実行は，個別の API 呼び出し操作に比べて，攻撃者が短時間で一連の操作を完了させる挙動を模擬する．

4.2 ネットワーク構成

検証環境の役割と関係を図 2 に示す．OAuth アプリと API 呼び出しを担う攻撃者ホスト，認可とブラウザ操作を行うユーザホスト，監査ログを取得，整理するログ監視ホストの 3 台を用意する．各ホストは外部通信用と管理用のインタフェースを分離した構成とする．

5. 評価

5.1 検証実験

検証実験は，検証実験用アプリを介して，ユーザ 1 およびユーザ 2 の Google Workspace アカウントに対して実施する．攻撃者の環境は，ローカルで起動した Python 製 Web サーバと，ngrok によるトンネリングを用いて外部からアクセス可能なリダイレクト URI を構築する．ユーザの環境は， 3.1 節の通り，API アクセスレベルが「制限なし」に設定された Google Workspace テナントを使用し，攻撃者が用意したアプリは Google による確認プロセスを経ていない未承認アプリとする．認可フローは Authorization Code Grant において PKCE [9] を併用しアクセストークンを取得する．

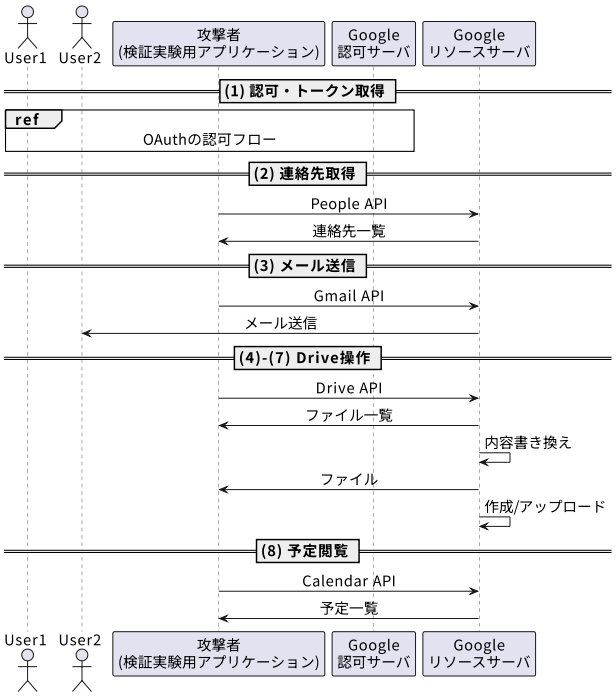


図 3: 検証実験のフロー

5.1.1 攻撃シナリオの実装詳細

3.3 節で定義したシナリオに基づき，単一の OAuth 認可フローで取得したアクセストークンを用いて，Workspace 上の複数サービスを横断的に悪用する一連の操作を実装する．要求するスコープは，連絡先閲覧（contacts.readonly），Gmail 送信（gmail.send），Drive 全体（drive），および予定閲覧（calendar.readonly）の 4 種である．

図 3 に示すように，次の (1)〜(8) の一連の操作を Python スクリプトで実行する．付録の図 A·1 に図 3 で示す検証実験のフローの詳細を記載する．各操作と，使用する API，対応するスコープの関係を表 1 に示す．

(1) ユーザ認可による認可とトークン取得
(2) 連絡先の取得
(3) 取得した連絡先へのメール送信
(4) Drive 上のファイル一覧・メタデータ取得
(5) Drive ファイル書き換え
(6) Drive ファイルのダウンロード（エクスポート）
(7) Drive フォルダ作成およびファイルのアップロード
(8) 予定の閲覧

© 2026 Information Processing Society of Japan

表 1: OAuth アプリによる実施操作と使用スコープ

操作	API	使用スコープ
(1) 認可・トークン取得	OAuth 2.0 認可コードフロー	（下記 4 スコープを一括要求）
(2) 連絡先取得	People API	`contacts.readonly`
(3) メール送信	Gmail API	`gmail.send`
(4) ファイル一覧・メタデータ取得	Drive API	`drive`
(5) Drive ファイル書き換え	Drive API	`drive`
(6) Drive ファイルのダウンロード（エクスポート）	Drive API	`drive`
(7) Drive フォルダ作成・ファイルのアップロード	Drive API	`drive`
(8) 予定閲覧	Calendar API	`calendar.readonly`

5.1.2 監査ログ収集方法

3.1 節で作成したユーザ 1 およびユーザ 2 を対象に，1 回の検証実験で生成された監査ログを取得する．監査ログの収集は，管理コンソール UI および Reports API を用いる．

管理コンソール UI 経由の収集は，管理コンソールの「監査と調査」ツールを用いる．ログソースをユーザのログイベント，Gmail ログイベント，Drive ログイベント，カレンダーログイベント，および Google Contacts のそれぞれに対して検索を行い，検索結果をスプレッドシートへエクスポートする．

Reports API 経由の取得は 30 秒間隔のポーリングで行い，新規イベントのみを JSONL 形式および JSON 形式で保存する．分析は，取得したイベント情報のうち，Reports API のレスポンスをそのまま保持した raw フィールドを用いる．raw フィールドは，イベントの詳細情報を含み，5.2.2 節における管理コンソール UI との表示項目を比較するために用いる．

実験者が，検証実験用アプリと同一の挙動を連絡先，Gmail，Drive，および Calendar の画面上で代替実施し，検証実験用アプリにおける監査ログ出力と比較する．以下，実験者による代替操作を「手動操作」と呼ぶ．

5.2 結果

検証実験のスクリプトの実行を開始してから，(8) 予定の閲覧が完了するまでに要した時間は，合計 10 回の試行において平均 22.1 秒である．

5.2.1 監査ログの記録有無

表 2 に 5.1.1 節の (1)〜(8) の操作について，Reports API 経由および管理コンソール UI 経由の双方で記録の有無を確認した結果を示す．表 2 の 8 操作は，いずれも検証実験用アプリによる API 呼び出しとして実行したものであり，表の 2 列は監査ログの収集経路の違いを表す．

8 操作のうち，(1)，(3)，(5)，(6)，および (7) の 5 操作は，両経路で記録された．一方，(2) 連絡先取得，(4) ファイル一覧・メタデータ取得，(8) 予定閲覧の 3 操作は，Reports API 経由・管理コンソール UI 経由のいずれにも記録されない．記録されない 3 操作は，いずれもデータの取得または閲覧のみを行う読み取り専用操作である点が共通する．

5.2.2 管理コンソール UI と Reports API の比較

検証実験用アプリの操作カテゴリごとに，管理コンソール UI 出力と Reports API 出力を比較し，記録項目および記録粒度の差異を明らかにする．

- **Gmail 送信**：管理コンソール UI 出力では 3 件のみが記録される一方，Reports API 出力では 6 件のイベントが記録される．件数に差はあるが，OAuth アプリの関与を示すアクターのアプリ情報は，管理コンソール UI 出力と Reports API 出力の両方で確認できる．
- **Drive ファイル書き換え・ダウンロード**：管理コンソール UI 出力と Reports API 出力は同一である．
- **フォルダ作成・txt アップロード・Google ドキュメント作成・一括アップロード**：管理コンソール UI 出力では，サブイベントは表示されず，操作カテゴリとして「作成」または「アップロード」の 1 行のみが表示される．一方，Reports API 出力には，主要イベント（create または upload，edit）に加え，change_user_access（ユーザのアクセス権限の変更），change_acl_editors（編集者のアクセス権限の変更），add_to_folder（フォルダへの追加）というサブイベント [10] が含まれる．

5.2.3 検証実験用アプリと手動操作の比較

管理コンソール UI 上の記録において，検証実験用アプリ経由の操作と手動操作の間で記録内容を比較し，検証実験用アプリの関与を識別可能なフィールドを明らかにする．

- **ログイン**：手動操作の記録は，ログインの種類が「Google のパスワード」である 1 件のみである．検証実験の記録は，同様の 1 件に加え，ログインの種類が「再認証」である記録を含む，計 2 件である．
- **Gmail 送信**：送信開始イベントに差異が存在する．手動操作では，アクターのアプリ情報は空欄，デバイスのセッション ID が記録され，クライアントの種類は「ウェブ」，OAuth プロジェクト ID は「0」である．検証実験では，アクターのアプリ情報に OAuth クライアント ID・アプリ名（gws-oauth-research2）・なりすま

© 2026 Information Processing Society of Japan

表 2: 検証実験用アプリの操作に対する監査ログの記録状況（✓: 記録あり，–: 記録なし）

操作	Reports API 経由	管理コンソール UI 経由
(1) 認可・トークン取得	✓	✓
(2) 連絡先取得	–	–
(3) メール送信	✓	✓
(4) ファイル一覧・メタデータ取得	–	–
(5) ファイル書き換え	✓	✓
(6) ダウンロード・エクスポート	✓	✓
(7) フォルダ作成・アップロード	✓	✓
(8) 予定閲覧	–	–

しフラグ（false）が記録される．デバイスのセッション ID は空欄，クライアントの種類は「API」，OAuth プロジェクト ID はクライアント ID の数字部分と一致する値である．以降のイベント（メールサーバからの送信，受信）に差異はない．

- **Drive ファイル書き換え**：アプリ ID・アプリ名は，検証実験の記録のみに存在する．加えて，1 件目のイベント種別は，手動操作で「表示」，検証実験で「編集」として記録される．
- **Drive ダウンロード**：アプリ ID・アプリ名・API メソッド（drive.files.export）は，検証実験の記録のみに存在する．
- **Drive フォルダ作成・txt アップロード・一括アップロード**：アプリ ID・アプリ名は，検証実験の記録のみに存在する．一括アップロードでは，フォルダの「表示」イベントが，手動操作の記録のみに存在する．
- **Google ドキュメント作成**：アプリ ID・アプリ名は，検証実験の記録のみに存在する．2 件目のイベントは，手動操作では「名前を変更」として記録され，古い値・新しい値の欄にファイル名の変更前後が記録される．検証実験では「編集」として記録され，アプリ ID・アプリ名が付与される．3 件目の「編集」イベントは，検証実験の監査ログのみに存在する．

また，ユーザ自身の画面に残る痕跡には，呼び出す API により差が見られた．Gmail API を介した送信は利用者の送信済みトレイに記録される一方，Drive 上のファイルのダウンロードは利用者の画面に痕跡を残さない．

6. 考察

6.1 サードパーティアプリ利用検知の考察

Google Workspace 管理コンソールの監査ログでは，OAuth 認可の成立時にイベントが記録される．イベントには，oauthClientId，applicationName，付与スコープの一覧が記録されており，管理者は認可日時，認可したユーザのアカウント，および認可対象の OAuth アプリの特定が可能である．管理コンソール UI のアプリ管理画面から，組織内で認可された全アプリの一覧が表示されるため，管理者は OAuth アプリがユーザにより新規に認可されたことが確認できる．

イベントの記録は，管理者が能動的に管理コンソール UI を確認する運用をしていない限りは，OAuth アプリに対するスコープの認可の発生を即時に把握することは困難である．また，監査ログからは OAuth アプリの利用は検知できるが，攻撃者による悪意のあるアプリであるかは判別できないため，管理者はアプリの利用を速やかに停止することができない．

一方，悪意のあるアプリの振る舞いは即座に完了せず，潜伏期間を経て実行される場合もある．管理コンソール UI のログ保存期間 [11] に依存せず，インシデント発生後の監査に Reports API 出力を用いることが有効であり，フォレンジック調査への活用が期待できる．

6.2 サードパーティアプリの悪意ある行動検知の考察

認可された OAuth アプリによる API 呼び出しは，drive edit，download，acl_change 等のイベントとして監査ログに記録され，各イベントには動作を行うアプリを示す originating_app_id が付与される．そのため，特定の OAuth アプリに紐づく操作履歴を，時系列で追跡することができる．

API 呼び出しは，付与スコープの範囲内であれば正規の権限行使として監査ログに記録されるため，通常の業務利用による操作と攻撃目的の操作を，監査ログの記述内容のみから区別する基準が存在しない．異常な操作をアラートとして検知するには，短時間の大量アクセスや，通常と異なる IP アドレスからのアクセスを，ユーザの平常時の利用パターンとの違いとして捉える仕組みが必要である．

6.1 節および 6.2 節で述べた検知可能な項目は，いずれも監査ログという事後出力に基づく議論である．しかし，検証実験のスクリプトの実行開始から終了までの一連の攻撃操作は，平均 22.1 秒で完了しており，管理者による検知と対応も同じ時間内に完了しなければ，攻撃の成立を未然に防ぐことはできない．事後解析を前提とした管理コンソール UI，および Reports API 監査ログ運用では，攻撃完了前の対応は困難である．

© 2026 Information Processing Society of Japan

6.3 対策の提案

5.2 節， 6.1 節，および 6.2 節で述べた検知の限界は，記録が残らない操作が存在すること，記録の粒度が収集経路により異なること，実行主体を識別可能な場合でも悪意ある操作かどうかは判別できないこと，および攻撃完了までの時間が短いことの 4 点に整理できる．管理者が現行の機能で講じられる対策として (1),(2), および (3) を示し，Google Workspace 自体の設計変更が必要な対策として (4) および (5) を本研究の結果に対応づけて述べる．

(1) ユーザからの申告制度の作成：5.2.3 節で示すように，Gmail API を介した送信は利用者の送信済みトレイに記録されるが，Drive のダウンロード（エクスポート）は利用者の画面に記録が残らない．Gmail 送信については，心当たりのない送信メールをユーザが申告できる制度を整備することで，ユーザ自身による早期発見を促す．一方，Drive ダウンロードはユーザによる発見可能性が低いため (4) で述べる管理者側のアラート整備が必要である．

(2) 認可時点での制御：5.2.1 節で示すとおり，Drive ファイル一覧の取得や Calendar の予定閲覧はいずれの経路にも記録されない．記録がなければ事後の検知は成立しないため，一覧取得や閲覧の操作に対しては認可の成立自体を防ぐことが対策となる．未承認のサードパーティアプリからのデータアクセスを制限する設定を有効化し，管理者が信頼済みに指定したアプリのみを認可する．また，要求スコープ単位で認可を判断し，Drive 全体（drive）のように閲覧範囲の広いスコープを要求するアプリには，用途を限定したスコープへの変更を求める．

(3) 監査を Reports API で行う：5.2.2 節で示すとおり，権限付与や ACL 変更を表すサブイベントは Reports API 出力にのみ含まれ，管理コンソール UI では「作成」または「アップロード」の 1 行に集約される．管理コンソール UI の閲覧のみに依存する運用では，共有範囲の変更を伴う操作を通常のファイル作成と区別できない．監査は Reports API を用い，サブイベントを含めて保存する．

(4) アプリ経由の操作を識別するアラート条件の設定：平常時の利用パターンからの逸脱を検出するアラートルールを設定する．短時間での大量ダウンロードや，同時刻に異なる IP アドレスからのアクセスなど，ユーザの通常利用との違いを捉える仕組みを構築することで，事後解析に頼らない早期の異常検知を可能にする．5.2.3 節で示す検証実験用アプリと手動操作の監査ログの違いを用いてアラート条件を設計する．

(5) アプリアクセスの自動無効化：5.2 節で示すとおり，一連の操作は平均 22.1 秒で完了する．管理者が通知を受けて手動で対応する運用では，被害の発生に間に合わない．また，監査ログの事後分析に期待できるのは，攻撃の阻止ではなく被害範囲の特定である．(4) の条件に合致した時点でアプリのアクセスを無効化し，発行済みトークンを失効させる処理を自動化する必要がある．

6.4 本研究の限界

本研究の結果は，特定の条件下での観測である．第一に，検証環境は Google Workspace Business Starter の単一テナントである．監査機能の提供範囲はエディションにより異なるため，本研究で記録されない操作が，上位のエディションでも同様に記録されないとは限らない．実際に Microsoft Exchange Online の事例 [3] では，強化ログの購入状況により被害組織の検知可否が分かれている．

第二に，対象は 2 つのユーザアカウント，1 つの OAuth アプリ，8 種類の操作，および合計試行回数が 10 回という限られた規模であり，他のアカウントやアプリでも同じ結果になるとは限らない．

第三に，監査ログの取得は 30 秒間隔のポーリングによる．記録の反映に遅延がある場合，本研究が記録されないとした操作に，遅れて記録が現れる可能性がある．

7. 関連研究との比較

本研究との詳細な比較を付録の表 A·1 に示す．

Michael ら [12] は，監査ログを要約・近似した際に攻撃の証拠としての価値がどれだけ保持されるかを定量化する指標を与えた．Michael らの対象は OS レベルのログにとどまる．SaaS の監査ログにおいて，第三者アプリ経由の操作がどこまで記録されるかという点は整理されていない．

Farooqi ら [13] は，Facebook 上の脆弱なサードパーティアプリから OAuth アクセストークンが流出し，評判操作サービスに悪用される実態をハニーポットを用いる実験により定量的に示している．しかし，検知はプラットフォーム運営者側の大規模な仕組みを前提としており，組織の管理者が監査ログのみを用いて濫用を検知できるかという観点は扱われていない．

Shah ら [14] は，侵害された組織内のユーザのアカウントのフォレンジック分析を通じて，攻撃者が長期間アカウント内に滞在する傾向を示し，非リアルタイム検知の場合も防御価値を持つと述べている．しかし，管理者が参照する監査ログ自体に操作の記録が残されているかというログの網羅性については検証の対象としていない．

Luo ら [15] は，複数の外部アプリやサービスを連携したプラットフォームにおいて，OAuth の認可サーバとクライアントの役割が逆転する構造に着目し，条件が揃えば被害者の操作一つでアカウント乗っ取りが成立することを実証している．しかし，攻撃の成立可否と防止策に焦点が置かれており，防御策導入以前に攻撃が成立した場合の監査ログを用いた検知可能性は検証の対象としない．

Wan ら [16] は，Google Workspace を含む共同作業基盤において，共同編集の機能を介してサードパーティアプリが管理者の設定した権限分離を回避しうることを示す．

© 2026 Information Processing Society of Japan

Zha ら [17] は，チームチャットサービスのサードパーティアプリ連携において権限昇格や情報漏洩につながる設計上の問題を報告している．いずれも権限の回避や昇格が成立するかに焦点があり，成立した操作が監査ログにどう記録されるかは扱わない．

8. まとめ

本研究は，組織内ユーザが Google 未承認の OAuth アプリの権限要求を認可した場合に，管理者が Google Workspace の監査ログを用いてどこまで攻撃活動を検知できるかを明らかにすることを目的とした．Google Workspace テナント上に自作の OAuth アプリを構築し，連絡先取得，メール送信，Drive 操作，および予定閲覧を含む 8 操作を 1 つの検証実験のスクリプトとして自動実行し，Reports API による監査ログ出力と管理コンソール UI による出力を取得し，監査ログを比較分析した．

分析の結果，監査ログには検証実験用アプリが利用されたことは示すが，操作内容自体を通常操作と区別する情報を含まないことを明らかにした．また，OAuth アプリ経由の操作とユーザによる手動操作を比較し，実行の主体が識別可能なフィールドを明らかにした．監査ログの記録範囲および検知速度の限界を踏まえ，管理者が現行の機能を用いて行える対策および，Google Workspace の設計の変更が必要な対策を提案した．

今後の課題は，コーディングエージェントやブラウザ操作エージェントが認可操作を代行する場合の認可フロー成立過程についての検証である．AI エージェントがユーザに代わり Web サービス上の操作を自律的に実行する際，ユーザの認識しない範囲で OAuth アプリへの認可を追加的に行う可能性がある．ユーザが個々の認可内容を把握しないまま権限が拡大するため，本研究で扱う攻撃シナリオとは異なる脅威が生じ得る．AI エージェント経由の認可成立過程と，認可操作の監査ログ上での記録可能性について検証する必要がある．

謝辞

本研究は JST 経済安全保障重要技術育成プログラム JPMJKP24K1 および JSPS 科研費 JP23K03847 の成果を含みます．

参考文献

[1] Google: Google Workspace, Google (online), available from ⟨https://workspace.google.com/intl/ja/?hl=ja⟩ (accessed 2026-08-09).

[2] Microsoft Threat Intelligence: Threat Actors Misuse OAuth Applications to Automate Financially Driven Attacks, Microsoft Security Blog (online), available from ⟨https://www.microsoft.com/en-us/security/blog/2023/12/12/threat-actors-misuse-oauth-applications-to-automate-financially-driven-attacks/⟩ (accessed 2026-08-18).

[3] Cyber Safety Review Board: Review of the Summer 2023 Microsoft Exchange Online Intrusion, Technical report, U.S. Department of Homeland Security (2024). Accessed 2026-08-13.

[4] Balash, D. G., Wu, X. and Grant, M. et al.: Security and Privacy Perceptions of Third-Party Application Access for Google Accounts, *31st USENIX Security Symposium (USENIX Security 22)*, USENIX Association, pp. 3397–3414 (2022).

[5] Google: Admin SDK: Reports API, Google (online), available from ⟨https://developers.google.com/workspace/admin/reports/reference/rest⟩ (accessed 2026-08-09).

[6] Hardt, D.: The OAuth 2.0 Authorization Framework, RFC 6749 (2012).

[7] Jones, M. and Hardt, D.: The OAuth 2.0 Authorization Framework: Bearer Token Usage, RFC 6750 (2012).

[8] Google: Authorize unverified third-party apps, Google Workspace (online), available from ⟨https://knowledge.workspace.google.com/admin/apps/authorize-unverified-third-party-apps⟩ (accessed 2026-08-18).

[9] Sakimura, N., Bradley, J. and Agarwal, N.: Proof Key for Code Exchange by OAuth Public Clients, RFC Editor (online), DOI: 10.17487/RFC7636 (accessed 2026-08-18).

[10] Google: Drive Audit Activity Events, Google Workspace (online), available from ⟨https://developers.google.com/workspace/admin/reports/v1/appendix/activity/drive⟩ (accessed 2026-08-18).

[11] Google: Data retention and lag times, Google Workspace (online), available from ⟨https://knowledge.workspace.google.com/admin/reports/data-retention-and-lag-times⟩ (accessed 2026-08-20).

[12] Michael, N., Mink, J. and Liu, J. et al.: On the Forensic Validity of Approximated Audit Logs, *Proceedings of the 36th Annual Computer Security Applications Conference*, Association for Computing Machinery, pp. 189–202 (2020).

[13] Farooqi, S., Zaffar, F. and Leontiadis, N. et al.: Measuring and mitigating oauth access token abuse by collusion networks, *Proceedings of the 2017 Internet Measurement Conference*, Association for Computing Machinery, pp. 355–368 (2017).

[14] Shah, N., Ho, G. and Schweighauser, M. et al.: A Large-Scale Analysis of Attacker Activity in Compromised Enterprise Accounts, arXiv (online), available from ⟨https://arxiv.org/abs/2007.14030⟩ (accessed 2026-08-18).

[15] Luo, K., Wang, X. and Fung, P. H. A. et al.: Universal Cross-app Attacks: Exploiting and Securing OAuth 2.0 in Integration Platforms, *34th USENIX Security Symposium (USENIX Security 25)*, USENIX Association, pp. 3221–3238 (2025).

[16] Wan, L., Yan, C. and Wang, Z. et al.: Beyond the Scope: Security Testing of Permission Management in Team Workspaces , *IEEE Transactions on Dependable and Secure Computing*, pp. 1–15 (online), DOI: 10.1109/TDSC.2026.3715679 (2026).

[17] Zha, M., Wang, J. and Nan, Y. et al.: Hazard Integrated: Understanding Security Risks in App Extensions to Team Chat Systems (2022).

© 2026 Information Processing Society of Japan

研究倫理

本研究の目的は，未承認 OAuth アプリへの認可後に管理者が監査ログで検知できる範囲と限界を整理し，注意喚起することであり，攻撃手法の新規提供ではない．検証は研究者が管理する検証実験用の Google Workspace テナント上でのみ実施し，実在組織や第三者の実データは対象としていない．第三者が同等の攻撃を再現できる手順や，トークン・認可コード・接続先等の機微情報は記載していない．

付　　録

A.1　検証実験のフロー

図 A·1 は検証実験の詳細なフローを示すシーケンス図である．要求するスコープは，連絡先閲覧（contacts.readonly），Gmail 送信（gmail.send），Drive 全体（drive），および予定閲覧（calendar.readonly）の 4 種である．以下の (1)〜(8) の一連の操作を Python スクリプトで実行する．

(1) ユーザ認可による認可とトークン取得：
本文中の図 1 の OAuth の認可フローに基づき，上記 4 スコープの認可を行いトークンを取得する．

(2) 連絡先の取得：
contacts.readonly スコープを用いて，ユーザ 1 に登録されているユーザ 2 の連絡先を取得する．

(3) 取得した連絡先へのメール送信：
gmail.send スコープを用いて，ユーザ 1 のアカウントからユーザ 2 へメールを送信する．

(4) Drive 上のファイル一覧・メタデータ取得：
drive スコープを用いて，ユーザ 1 が Drive 内に保有するファイルの情報を取得する．

(5) Drive ファイル書き換え：
drive スコープを用いて，ユーザ 1 の Drive 内のファイルを書き換える．

(6) Drive ファイルのダウンロード（エクスポート）：
drive スコープを用いて，ユーザ 1 の Drive 内に保有するファイルを PDF 形式で攻撃者サーバへダウンロードする．

(7) Drive フォルダ作成およびファイルのアップロード：
drive スコープを用いて，ユーザ 1 の Drive にフォルダおよびファイルをアップロードする．

(8) 予定の閲覧：
calendar.readonly スコープを用いて，ユーザ 1 のカレンダーに登録されている予定を取得する．

A.2　関連研究の詳細な比較

「文献」は該当研究の著者名を示す．「研究内容」は各研究が実施した内容の概要，「解析対象」は各研究が扱うデータまたはシステム，「目的」は各研究が明らかにしようとした点をそれぞれ示す．

「監査ログ分析」は，各研究が監査ログを直接の解析対象としているかを示す．「SaaS 対象」は，解析対象が SaaS 上で提供されるサービスであるかを示す．「サードパーティアプリの関与」は，脅威モデルまたは攻撃手法にサードパーティアプリの関与が含まれるかを示す．

表中の ✓ は該当，–は非該当を表す．

© 2026 Information Processing Society of Japan

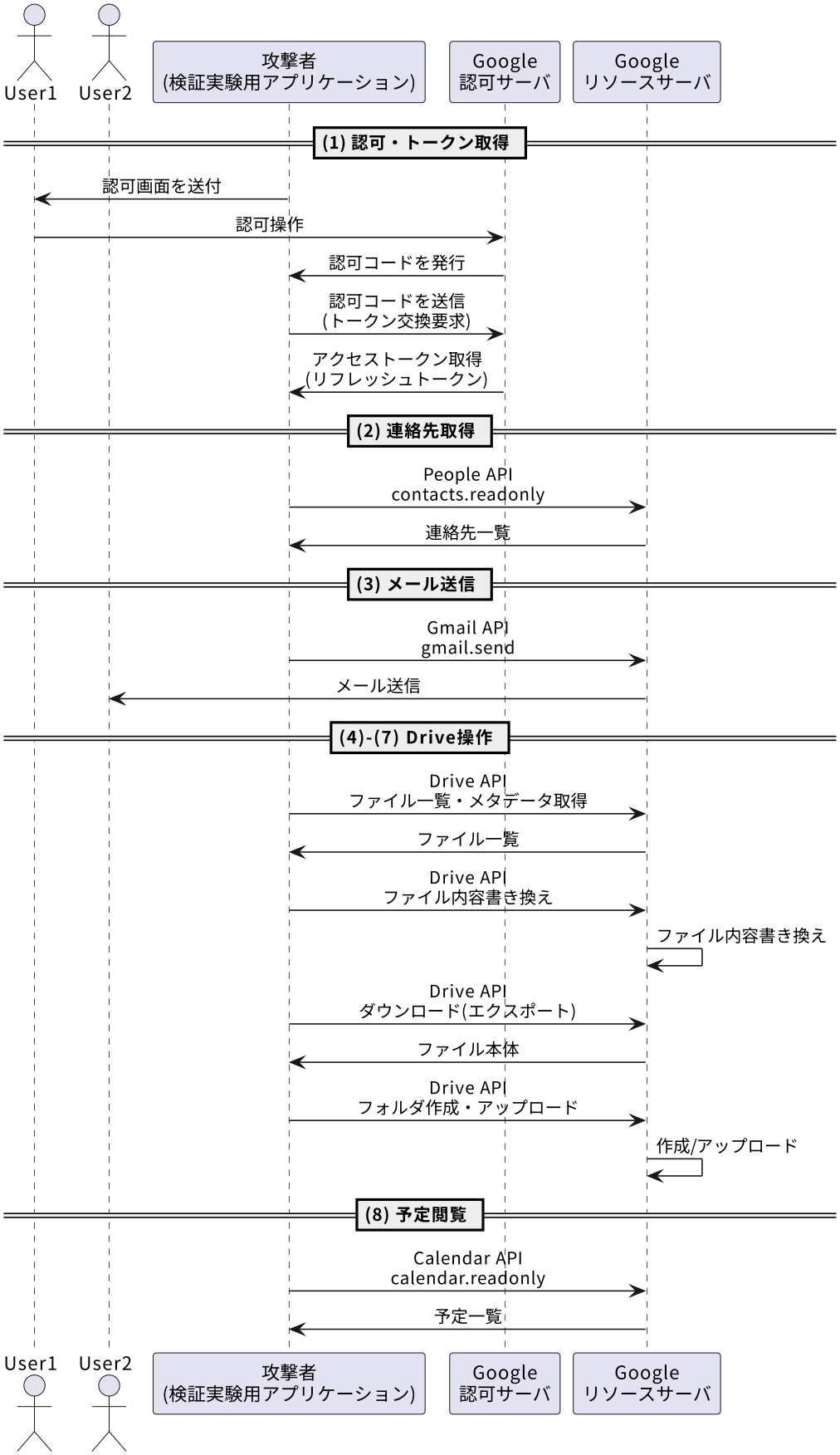


図 A·1: 検証実験のフロー詳細

© 2026 Information Processing Society of Japan

表 A·1: 関連研究の比較（✓: 該当，–: 非該当）

文献	研究内容	解析対象	目的	監査ログ分析	SaaS 対象	サードパーティアプリの関与
本研究	管理コンソールでの PoC 検証，監査ログの記録範囲の確認	Google Workspace の監査ログ	サードパーティアプリの利用と悪用検知	✓	✓	✓
Michael ら [12]	近似監査ログの証拠価値の定量化指標の提案，近似手法の評価	OS レベルの監査ログ (システムコール記録)	攻撃の証拠の残存度の明確化	✓	—	—
Farooqi ら [13]	おとりアカウントによる観測，OAuth トークン流出と悪用の定量化	Facebook 上のサードパーティアプリと OAuth トークン	トークン濫用の実態把握と対策	—	—	✓
Shah ら [14]	侵害アカウントの操作記録の痕跡調査手法の考案	侵害された企業アカウントの操作履歴	攻撃者の活動傾向の明確化	✓	✓	—
Luo ら [15]	OAuth 役割逆転構造への着目，アカウント侵害成立の検証	連携基盤における OAuth 認可の流れ	攻撃成立条件の明確化	—	✓	✓
Wan ら [16]	権限管理の仕組みの分析，アドオンによる権限分離回避の検証	チームワークスペースのアドオン権限管理 API	権限昇格の発生条件の明確化	—	✓	✓
Zha ら [17]	外部アプリの導入から運用までの分析，構造上の問題の報告	社内コミュニケーションサービスの外部アプリ連携 API	権限昇格と情報漏えいリスクの明確化	—	✓	✓

© 2026 Information Processing Society of Japan